

12. Further Abstraction Technique

Preface

Main concepts discussed in this chapter:

- abstract classes
- interfaces

This chapter explains how Java uses **abstract classes** and **interfaces** to support **abstraction**, **polymorphism**, and **multiple inheritance of behavior**.

Example 12.1

```
// A simple example that covers this chapter

abstract class Animal {
    public abstract void makeSound();
}

class Dog extends Animal {
    public void makeSound() {
        System.out.println("Bark");
    }
}

interface Movable {
    void move();
}

class RobotDog extends Dog implements Movable {
    public void move() {
        System.out.println("Run");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new RobotDog();
        a.makeSound();

        Movable m = (Movable) a;
        m.move();
    }
}
```

Example 12.2

```
interface Flyable {
    void fly();
}

abstract class Bird {
    abstract void eat();
}

class Sparrow extends Bird implements Flyable {
    public void eat() {
        System.out.println("Eat seeds");
    }

    public void fly() {
        System.out.println("Fly in the sky");
    }
}
```

12.1 Abstract Method

Definition

An **abstract method** is a method **without a body**.
It only gives a method **name and signature**.

Features

- Declared using the keyword `abstract`
- Ends with `;`
- Must be implemented by subclasses

Example

```
abstract class Animal {
    public abstract void makeSound();
}
```

- `makeSound()` has no body
- The class does not say *how* the sound is made

Remark

- Abstract methods describe **what to do**, not **how to do it**
-

12.2 Abstract Class

Definition

An **abstract class** is a class that **cannot be instantiated**. It may contain **abstract methods** and **concrete methods**.

Features

- Declared with keyword `abstract`
- Can have fields and constructors
- Can have abstract and non-abstract methods

Example

```
abstract class Animal {  
    public abstract void makeSound();  
  
    public void sleep() {  
        System.out.println("Sleeping");  
    }  
}
```

- `Animal` cannot be created using `new`
- Subclasses must implement abstract methods

Remark

- Abstract classes are used as **base classes**
-

12.3 Concrete Class

Definition

A **concrete class** is a class that **has no abstract methods**. It can be instantiated.

Features

- Implements all inherited abstract methods
- Can create objects from it

Example

```
class Dog extends Animal {  
    public void makeSound() {  
        System.out.println("Bark");  
    }  
}
```

- `Dog` implements `makeSound()`
- `Dog` is concrete

Remark

- Only concrete classes can create objects
-

12.4 Abstract Subclass

Definition

An **abstract subclass** is a subclass that **does not implement all abstract methods**.

Features

- Still declared as `abstract`
- May leave abstract methods unimplemented

Example

```
abstract class WildAnimal extends Animal {  
    // makeSound() not implemented  
}
```

- `WildAnimal` is still abstract

Remark

- Abstract subclasses delay implementation
-

12.5 Multiple Inheritance

Definition

Multiple inheritance means inheriting from **more than one parent**.

Features

- Java does **not** allow multiple inheritance of classes
- Java allows multiple inheritance using interfaces

Example

```
class RobotDog extends Dog implements Movable, Flyable {  
    public void move() {  
        System.out.println("Move");  
    }  
  
    public void fly() {  
        System.out.println("Fly");  
    }  
}
```

- One class + many interfaces is allowed

Remark

- This avoids ambiguity and complexity
-

12.6 Interface (Java Construct)

Definition

An **interface** defines a set of methods **without implementation**.

Features

- Declared using `interface`
- Methods are `public` and `abstract` by default
- No instance fields (only constants)

Example

```
interface Movable {  
    void move();  
}
```

- No method body
- Only behavior specification

Remark

- Interfaces define **capabilities**
-

12.7 implements Keyword

Definition

The keyword `implements` is used when a class **implements an interface**.

Features

- A class must implement all interface methods
- One class can implement multiple interfaces

Example

```
class RobotDog implements Movable {  
    public void move() {  
        System.out.println("Run");  
    }  
}
```

- `RobotDog` provides the method body

Remark

- `extends` → class inheritance
 - `implements` → interface inheritance
-

12.8 Abstract Class vs Interface

12.8.1 Abstract Class

Features

- Can have fields
- Can have constructors
- Can have concrete methods

12.8.2 Interface

Features

- No instance fields
- No constructors
- Supports multiple inheritance

Remark

- Use **abstract class** for *is-a relationship with shared code*
- Use **interface** for *shared behavior*